PROJET FINAL · PYTHON AVANCÉ · ING3

Analyser 12 Go de logs sans faire exploser la RAM

Engine de traitement et d'analyse de flux de données sécurisé

12,1 Go	69,3 M	210 834	48
de logs générés	de lignes analysées	alertes de sécurité	attaquants démasqués

L'ÉQUIPE

NAYME Yanis

GUEYE Babacar Mansour

CLABAU Brandon

LYOUBI Mehdi

Enseignant : JOUD HANAY

https://github.com/Babacar2804/Projet-PythonSecure-log-anlysis

Soutenance · 11/11/2026

1 Le défi et nos données

Un jeu de données réaliste, varié… et volontairement sale

Le sujet : construire en Python une plateforme en ligne de commande qui **ingère, nettoie et analyse** des journaux de logs volumineux, **sans jamais les charger entièrement en mémoire**. Pour la tester, nous avons écrit un générateur qui simule 30 jours d'activité d'un site e-commerce et de son infrastructure.

12,1 Go
écrits en 75 s par 7 processus en parallèle

69,3 M
lignes réparties en 12 fichiers

17 538
adresses IP différentes

11
services : API, paiement, base de données, VPN…

Trois formats de lignes bien structurés

data/raw/logs-00-20261001-0000.log

```
# applicatif (80 %) : 7 champs séparés par |
2026-10-01T08:15:32.481Z | INFO | api-gateway | 10.0.3.17:51432/TCP | user=u1042 | event=http_request | {...}
# JSON (10 %)
{"ts":"2026-10-01T08:15:32Z","lvl":"error","svc":"payment-service","client":{"ip":...},"code":502}
# pare-feu (10 %) : champs positionnels
FW 2026-10-01T00:20:12.423Z DROP TCP 102.109.77.236 27409 10.0.100.29 22 DENY-SSH-EXT
```

Des données variées

cycle jour / nuit · week-ends calmes · pannes · DDoS · déploiements · disque plein · force brute · injections SQL · XSS · scans de ports · botnet · IPv6

Utilisateurs selon une loi de Zipf, profil horaire réaliste, incidents planifiés : les analyses ont vraiment quelque chose à trouver.

Environ 5 % de lignes abîmées

`err · FATAL · Warning`	réparé
`USER_1042 · U-1042 · null`	réparé
`01/10/2026 · epoch · sans fuseau`	réparé
`{'status':'500'} · requÃªte`	réparé
`\x00 · \x1b[31m · \ufeff · \r`	réparé
`doublons · vides · stack traces`	supprimé
`999.12.4.1 · 2026-13-45 · tronqué`	rejeté

Comment fonctionne le générateur (generator.py)

Pools

Populations fixes : 5 000 utilisateurs, ~17 000 IP, 60 attaquants, un botnet. Même graine = mêmes IP dans tous les fichiers.

Scénarios

Pannes, DDoS, déploiements et disque plein planifiés sur les 30 jours, avec un profil horaire réaliste.

NoiseInjector

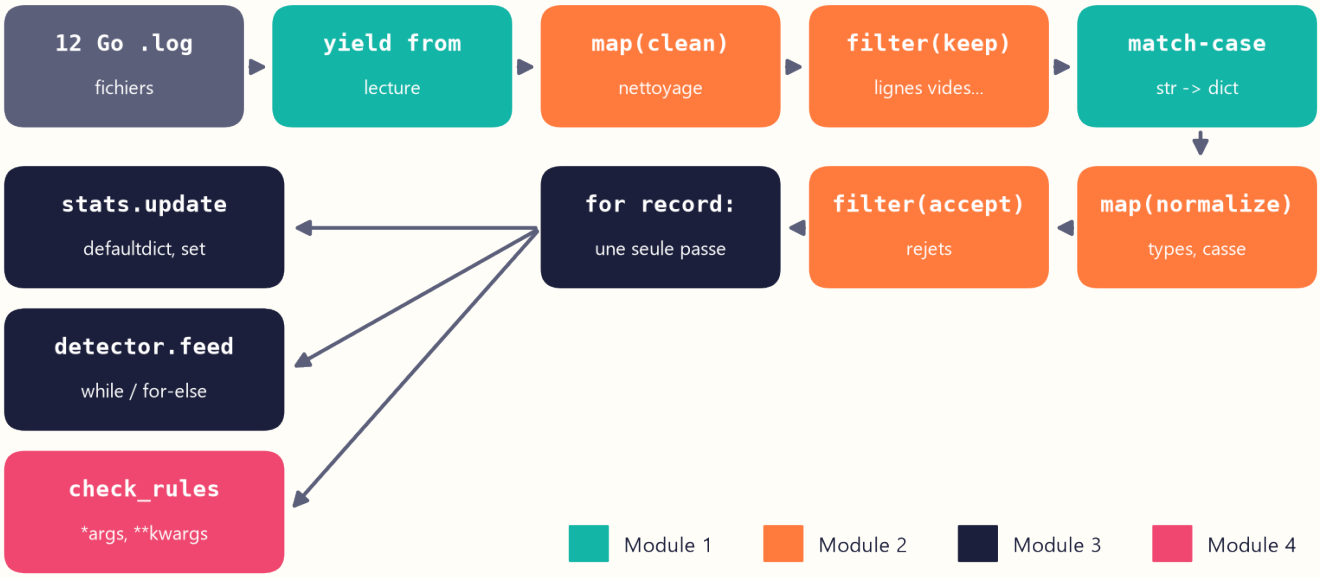
Abîme ~5 % des lignes : un match-case choisit le défaut (casse, date, IP, JSON, encodage…).

En parallèle

ProcessPoolExecutor : un processus par fichier, 12 fichiers écrits en même temps.

2 L'architecture

Une chaîne de générateurs : une ligne entre, une ligne sort, rien n'est stocké



Le pipeline en 8 lignes (main.py)

main.py

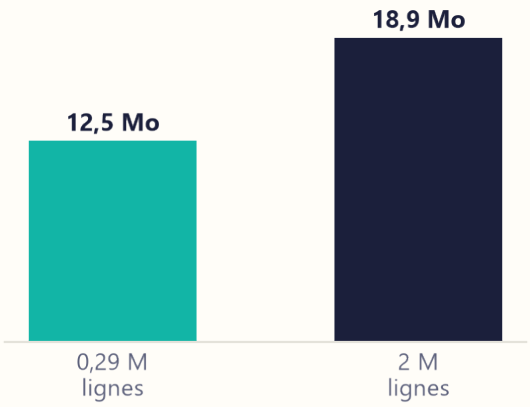
```
lines   = read_sources(files)               # yield from
lines   = map(clean_raw_line, lines)        # nettoyage du texte
lines   = filter(tracker.keep_line, lines)  # vides, #, traces
records = map(parse_line, lines)            # match-case -> dict
records = map(normalize_record, records)    # types, casse
records = filter(tracker.accept, records)   # rejets
for record in records:                      # RIEN n'est lu avant cette ligne
    stats.update(record); detector.feed(record); check_rules(record)
```

À RETENIR

map, **filter** et **yield** sont **paresseux** : ils ne calculent rien tant que la boucle **for** ne demande pas l'élément suivant. Chaque ligne traverse toute la chaîne, est comptée, puis oubliée. On ne garde que des compteurs : la mémoire dépend du nombre d'IP ou d'utilisateurs, **pas de la taille des fichiers**.

Le **même pipeline** sert au mode temps réel : seule la source change (`follow_log_file`, comme `tail -f`).

Mémoire mesurée



3 Le grand nettoyage

map + filter + match-case : réparer tout ce qui peut l'être, écarter le reste

68,3 M	1,3 M	0,5 M	1,4 %
enregistrements valides en sortie	corrections appliquées (lignes sauvées)	lignes supprimées (vides, doublons…)	seulement de pertes au total

Avant / après : une vraie ligne passée dans le pipeline

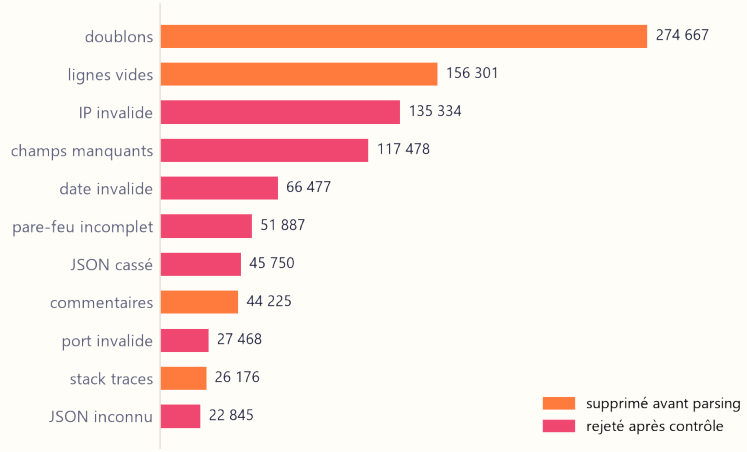
entrée

```
# ligne brute, telle qu'elle sort du générateur
01/10/2026 08:15:32 | err | API_GATEWAY | 88.12.34.56:51432 | user=USER_7 | event=http_request |
    {'status':'500','latency_ms':-1}
```

▼ parse_line puis normalize_record ▼

timestamp	2026-10-01 08:15:32+00:00	date réécrite en ISO, UTC
level	ERROR	« err » normalisé (match-case)
service	api-gateway	casse et « _ » corrigés
connection	('88.12.34.56', 51432, 'TCP')	tuple immuable, protocole TCP par défaut
user	u7	« USER_7 » ramené à « u7 »
payload	{'status': 500, 'latency_ms': None}	guillemets et latence corrigés

Ce qui a été écarté, et pourquoi



LE SAVIEZ-VOUS ?

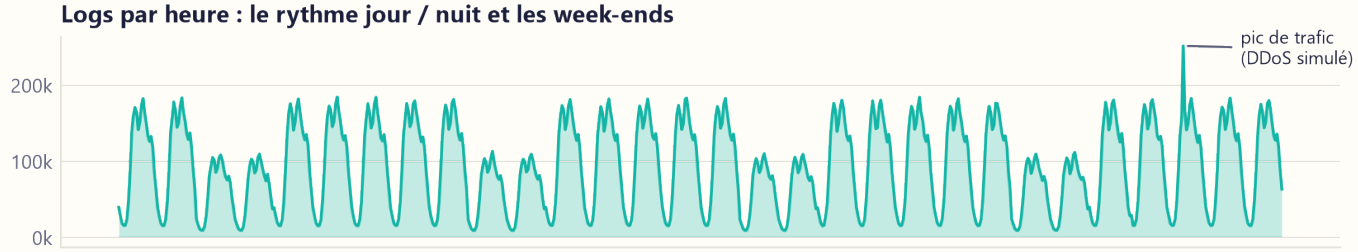
Chaque décision est **comptée par cause** : `filter` reçoit des méthodes (`tracker.keep_line`, `tracker.accept`) qui gardent ou écartent la ligne **et** notent pourquoi.

Les doublons sont détectés en comparant à la ligne précédente : un `set` de 69 M de lignes ferait exploser la RAM.

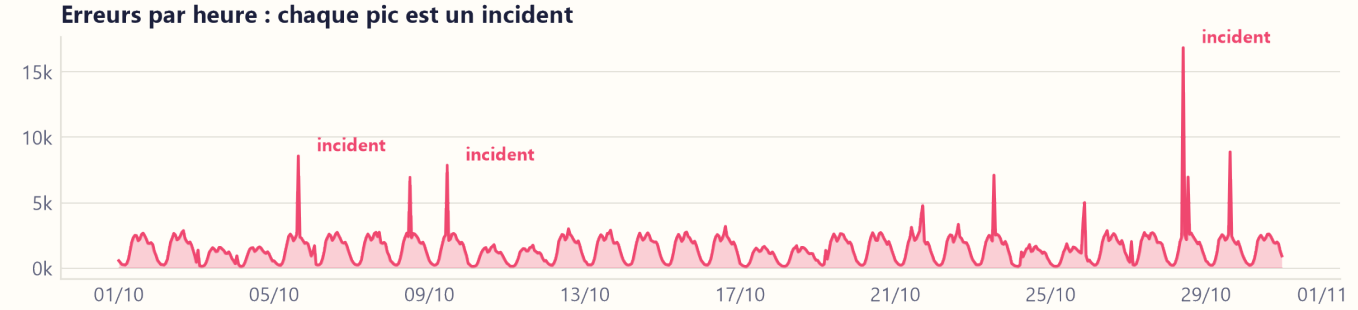
4 Ce que révèlent les données

defaultdict, set et frozenset : 68 millions de lignes résumées en quelques compteurs

Logs par heure : le rythme jour / nuit et les week-ends



Erreurs par heure : chaque pic est un incident



190
IP de la liste noire actives sur 490 (actives & blacklist)

300
IP listées jamais vues (blacklist - actives)

30
scanners : 15 ports visés ou plus

180 896
flux bloqués (IP, port, protocole)

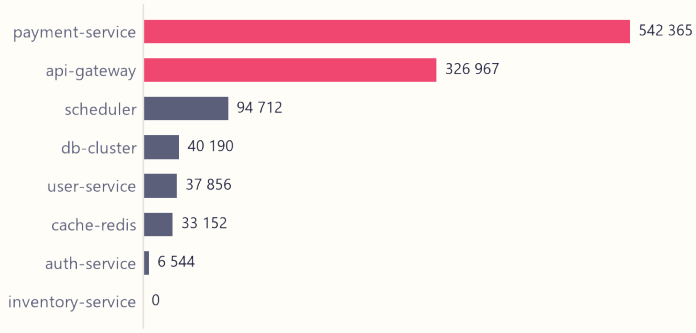
TUPLES

Le **tuple** (IP, Port, Protocole) est immuable, donc il peut servir de **clé de dictionnaire** : `blocked_flows[(ip, port, proto)] += 1`. En regroupant ces clés par IP, on repère les scans de ports.

ENSEMBLES

La liste noire est un **frozenset** : personne ne peut la modifier pendant l'analyse. Les opérations **| & - ^** comparent en une ligne les IP actives et les IP listées.

Erreurs par service : les pannes ressortent



aggregation.py

```
match record:
    case {"level": "ERROR" | "CRITICAL",
          "ip": ip}:
        self.error_ips.add(ip)
        self.error_timeline[hour] += 1
```

Un seul motif **match** vérifie le niveau, capture l'IP et alimente un **set** (IP uniques en erreur) et un **defaultdict** (erreurs par heure).

5 Sécurité : détecter les attaques

Une fenêtre glissante, un moteur de règles à arguments variables, et du temps réel

Force brute : la boucle qui tranche

```python
# detection.py
for moment, user, ok in tentatives:
    if not ok:
        consecutive += 1
        continue
    if consecutive >= 5 and récent:
        compromis = user
        break           # attaque réussie
    consecutive = 0
else:
    statut = "bloquée"  # aucun break
```

Pendant la lecture, une **while** purge une fenêtre glissante de 2 minutes par IP. Le **else** de la boucle ne s'exécute que si elle va au bout **sans break** : l'attaque a échoué.

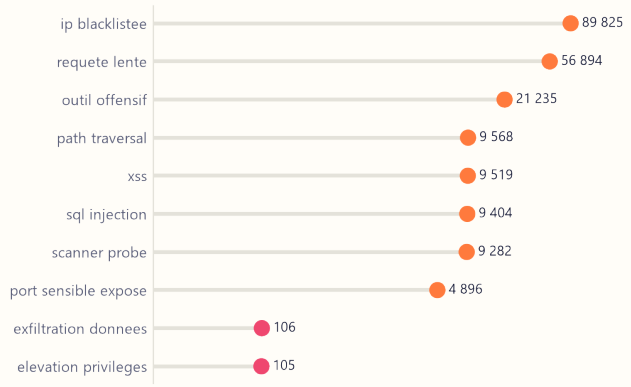
Résultat sur les 12 Go

60
IP attaquantes détectées

48
ont réussi à entrer

105
élévations de privilèges confirment

Le moteur de règles : apply_security_rules(*args, **kwargs)




```python
# rules.py
check = apply_security_rules(
    rule_sql_injection, rule_xss, ...,   # *args
    blacklist=ips, max_latency_ms=5000)  # **kwargs
for alert in check(record): ...
```

La fonction maître **renvoie une fonction** (fermeture). Une règle = une fonction ; une règle qui plante est isolée par **try / except / else**.

```
# démo temps réel
$ python main.py live
[live] 2,357 lignes lues, 7 alertes
[HIGH] xss  217.135.14.178 javascript:...
[FORCE BRUTE] 92.112.165.218 : 5 échecs
```

9 fonctions de règles, 10 types d'alertes

élévation de privilèges · exfiltration · injection SQL · XSS · path traversal · IP en liste noire · port sensible exposé · outil offensif · scanner (/.env, /wp-admin) · requête lente

CRITICAL · HIGH · MEDIUM · LOW

6 Qualité et barème

Chaque critère, et l'endroit exact où le montrer dans le code

Environnement & livrables	venv, requirements.txt (pip freeze), .py uniquement, rapport PDF, Git	2 pts
Mémoire & générateurs	yield, yield from, follow_log_file ; ~19 Mo pour 2 M de lignes	4 pts
Structures de données	set, frozenset, defaultdict, tuple clé de dict, match-case	4 pts
Programmation fonctionnelle	map, filter, lambda, méthodes passées à filter, fonction renvoyée	4 pts
Règles & introspection	apply_security_rules(*args, **kwargs), globals() / locals()	3 pts
Qualité & robustesse	PEP 8, mypy 0 erreur, 75 tests, try/except/else, for-else	3 pts

75
tests pytest, tous verts

0
erreur mypy sur code et tests

0
écart PEP 8 (pycodestyle)

69 204
lignes analysées par seconde

En résumé

CONCLUSION

Un moteur qui traite **12 Go de logs hétérogènes et dégradés** avec quelques dizaines de mégaoctets de mémoire, répare l'essentiel des défauts, retrouve les incidents et les attaques injectés, et fonctionne aussi **en temps réel**. Pistes d'amélioration : analyse parallèle par fichier, séparation des campagnes d'attaque d'une même IP.

DÉMO `python main.py demo` `python main.py live` `python -m pytest`

Code et rapport complet : https://github.com/Babacar2804/Projet-PythonSecure-log-anlysis